NICS 清華大學

Tsinghua University

Modern Computer Architecture Fall 2025

Lecture 09_2. Bus & I/O Systems

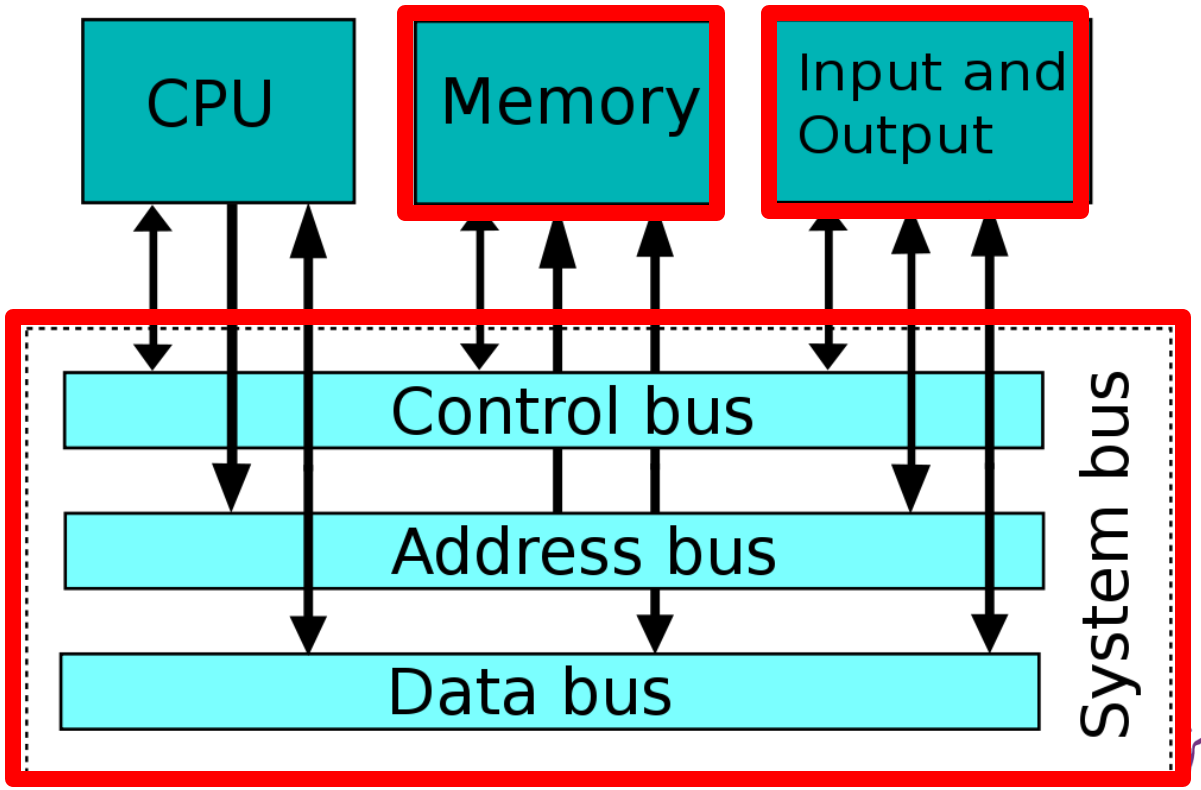
Yongpan Liu

ypliu@tsinghua.edu.cn

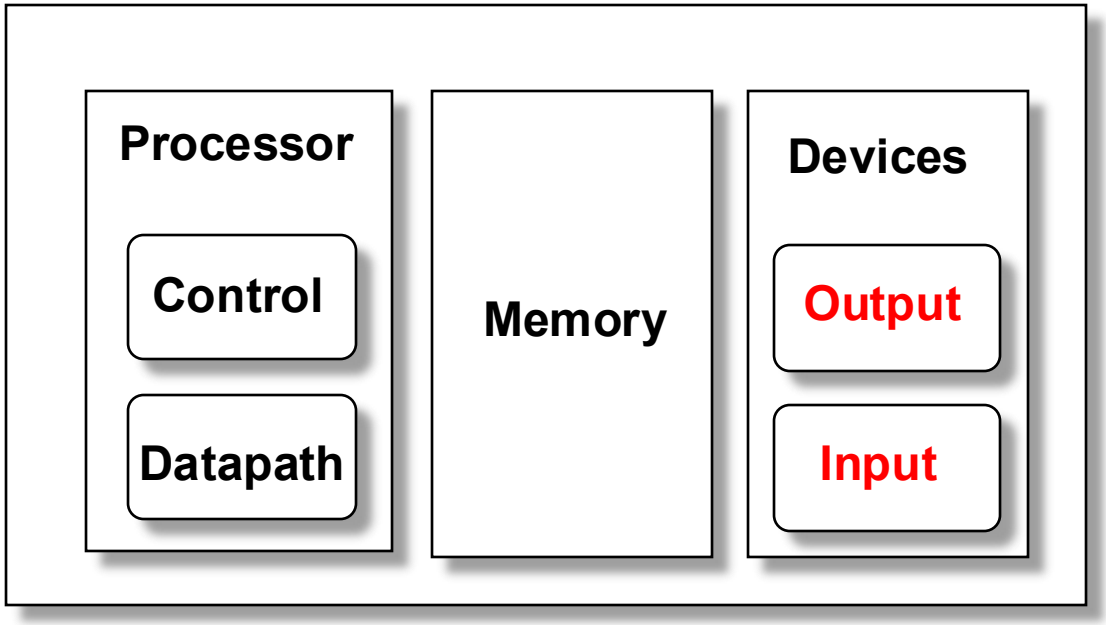
2025-Nov-26

Review: Performance Bottleneck

- Processor
- Memory
- Interconnection/Devices
- Software



Review: Major Components of a Computer



- Important metrics for an I/O system
  - Performance
  - Expandability, diversity / heterogeneity
  - Reliability
  - Cost, size, weight, power
  - Security

Input and Output Devices

- I/O devices are incredibly diverse with respect to
  - Behavior – input, output or storage
  - Partner – human or machine
  - Data rate – the peak rate at which data can be transferred between the I/O device and the main memory or processor

Device	Behavior	Partner	Data rate (Mb/s)
Keyboard	input	human	0.0001
Mouse	input	human	0.0038
Laser printer	output	human	3.2000
Graphics display	output	human	**800.0000-8000.0000**
Network/LAN	input or output	machine	100.0000-1000.0000
Magnetic disk	storage	machine	240.0000-2560.0000

I/O Performance Measures

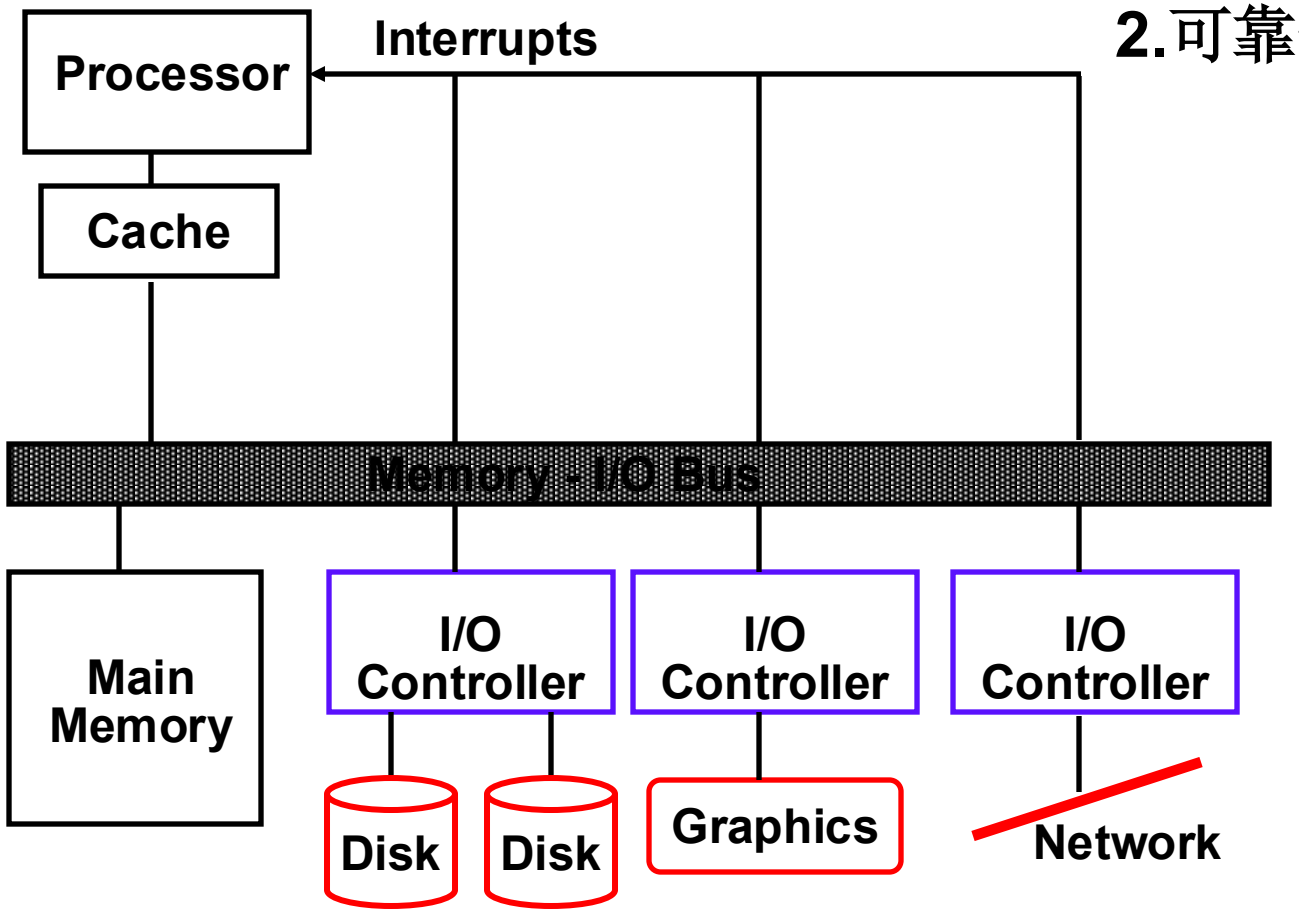
- I/O bandwidth (throughput) – amount of information that can be input (output) and communicated across an interconnect (e.g., a bus) to the processor/memory (I/O device) per unit time
  1. How much data can we move through the system in a certain time?
  2. How many I/O operations can we do per unit time?

- I/O response time (latency) – the total elapsed time to accomplish an input or output operation
  - An especially important performance metric in real-time systems

- throughput =? response times

A Typical I/O System

1.扩展性问题
2.可靠性问题

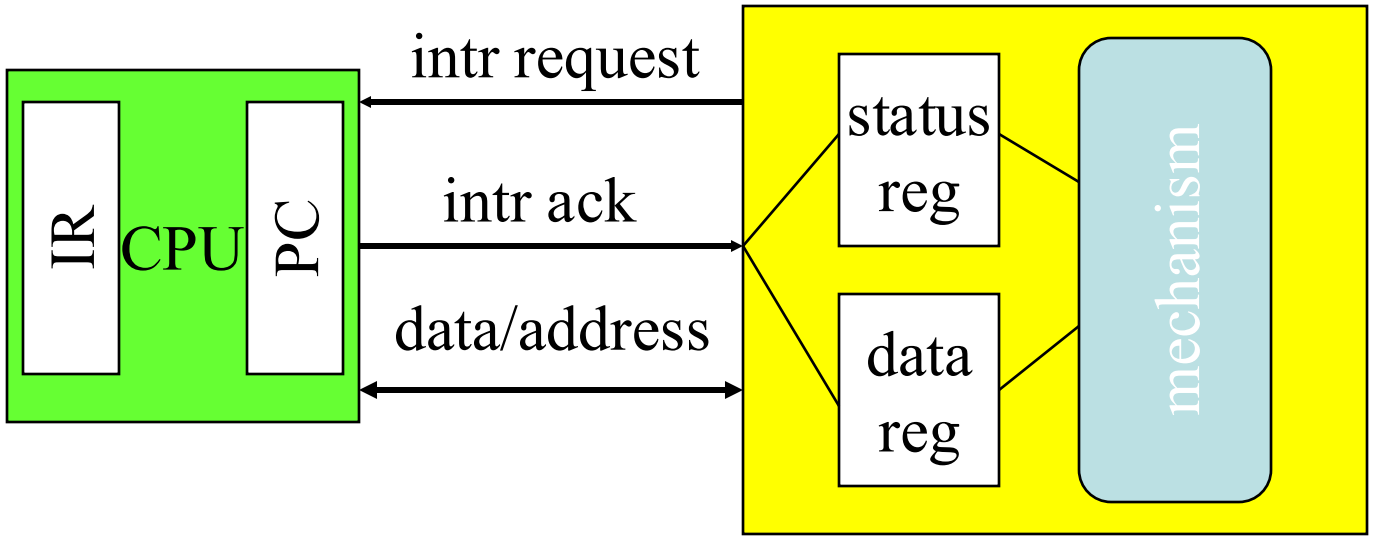


Interrupt I/O

- Busy/wait is very inefficient.
  - CPU can't do other work while testing device.
  - Hard to do simultaneous I/O.

- Interrupts allow a device to change the flow of control in the CPU.
  - Causes subroutine call to handle device.

Interrupt interface



Interrupt behavior

- Based on subroutine call mechanism.
- Interrupt forces next instruction to be a subroutine call to a predetermined location.
  - Return address is saved to resume executing foreground program.

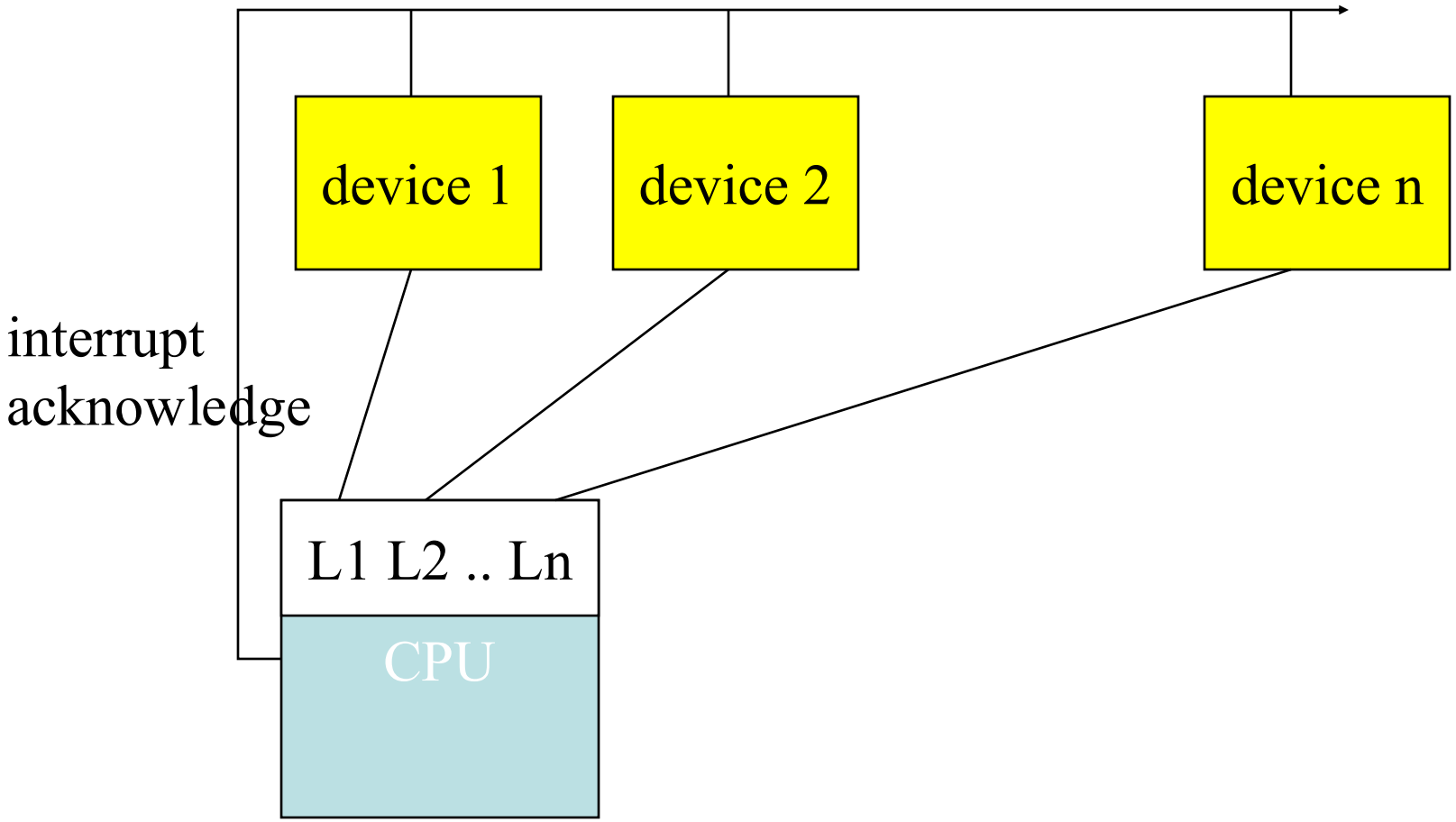
Interrupt physical interface

- CPU and device are connected by CPU bus.
- CPU and device handshake:
  - device asserts interrupt request;
  - CPU asserts interrupt acknowledge when it can handle the interrupt.

Priorities and vectors

- Two mechanisms allow us to make interrupts more specific:
  - Priorities determine what interrupt gets CPU first.
  - Vectors determine what code is called for each type of interrupt.
- Mechanisms are orthogonal: most CPUs provide both.

Prioritized interrupts

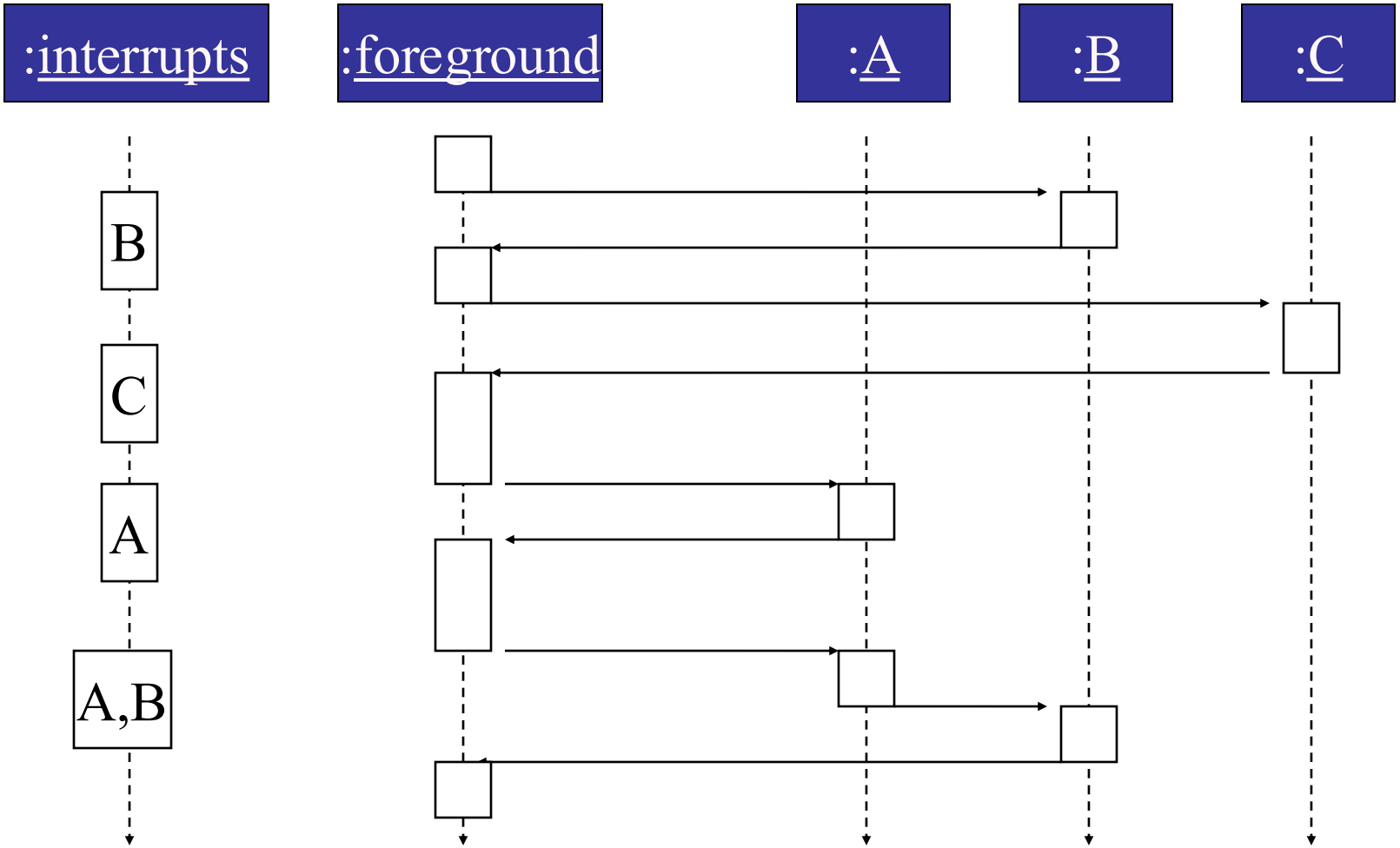


Interrupt prioritization

- Masking: interrupt with priority lower than current priority is not recognized until pending interrupt is complete.

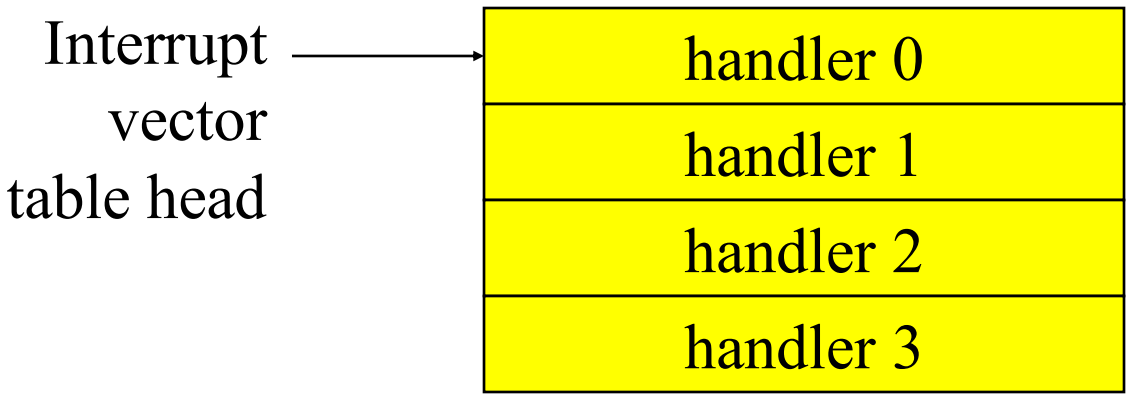
- Non-maskable interrupt (NMI): highest-priority, never masked.
  - Often used for power-down.

Example: Prioritized I/O

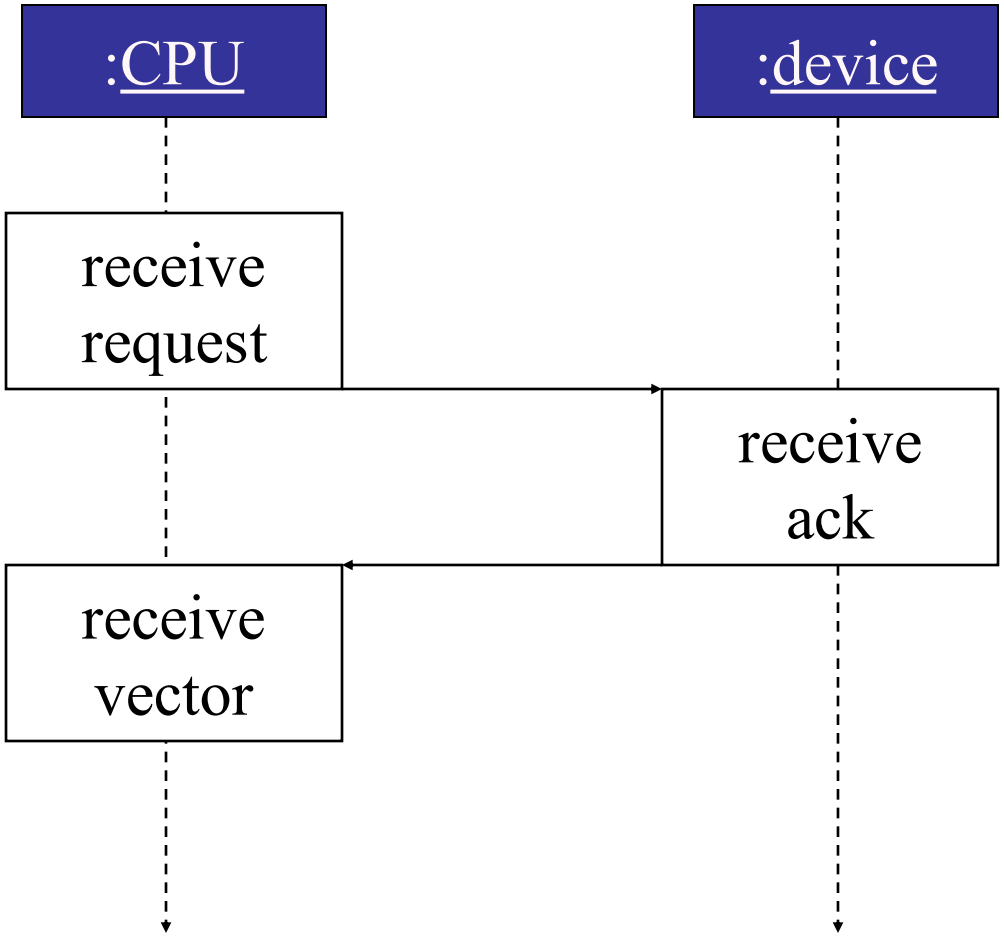


Interrupt vectors

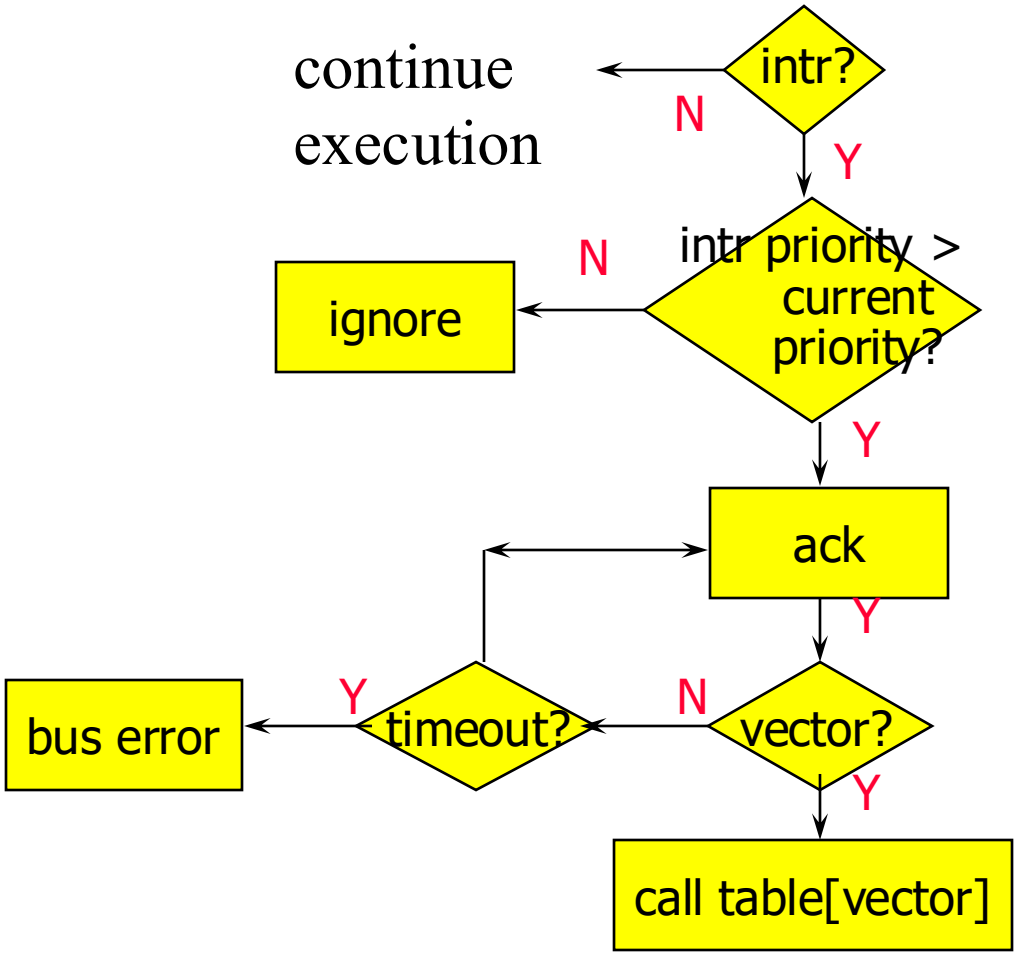
- Allow different devices to be handled by different code.
- Interrupt vector table:



Interrupt vector acquisition



Generic interrupt mechanism



Assume priority selection is handled before this point.

Interrupt sequence

- CPU acknowledges request.
- Device sends vector.
- CPU calls handler.
- Software processes request.
- CPU restores state to foreground program.

Sources of interrupt overhead

- Handler execution time.
- Interrupt mechanism overhead.
- Register save/restore. （Minimize register usage）
- Pipeline-related penalties.
- Cache-related penalties.

I/O System Performance

- Designing an I/O system to meet a set of bandwidth and/or latency constraints means

1. Finding the weakest link in the I/O system – the component that constrains the design
   - The processor and memory system ?
   - The underlying interconnection (e.g., bus) ?
   - The I/O controllers ?
   - The I/O devices themselves ?
2. (Re)configuring the weakest link to meet the bandwidth and/or latency requirements
3. Determining requirements for the rest of the components and (re)configuring them to support this latency and/or bandwidth

I/O System Performance Example

- A disk workload consisting of 64KB reads and/or writes where the user program executes 200,000 instructions per disk I/O operation and
  - a processor that sustains 3 billion instr/s and averages 100,000 OS instructions to handle a disk I/O operation

The maximum disk I/O rate (# I/O's/s) of the processor is

$$\frac{\text{Instr execution rate}}{\text{Instr per I/O}} = \frac{3 \times 10^9}{(200 + 100) \times 10^3} = 10{,}000 \text{ I/O's/s}$$

  - a memory-I/O bus that sustains a transfer rate of 1000 MB/s

Each disk I/O reads/writes 64 KB so the maximum I/O rate of the bus is

$$\frac{\text{Bus bandwidth}}{\text{Bytes per I/O}} = \frac{1000 \times 10^6}{64 \times 10^3} = 15{,}625 \text{ I/O's/s}$$

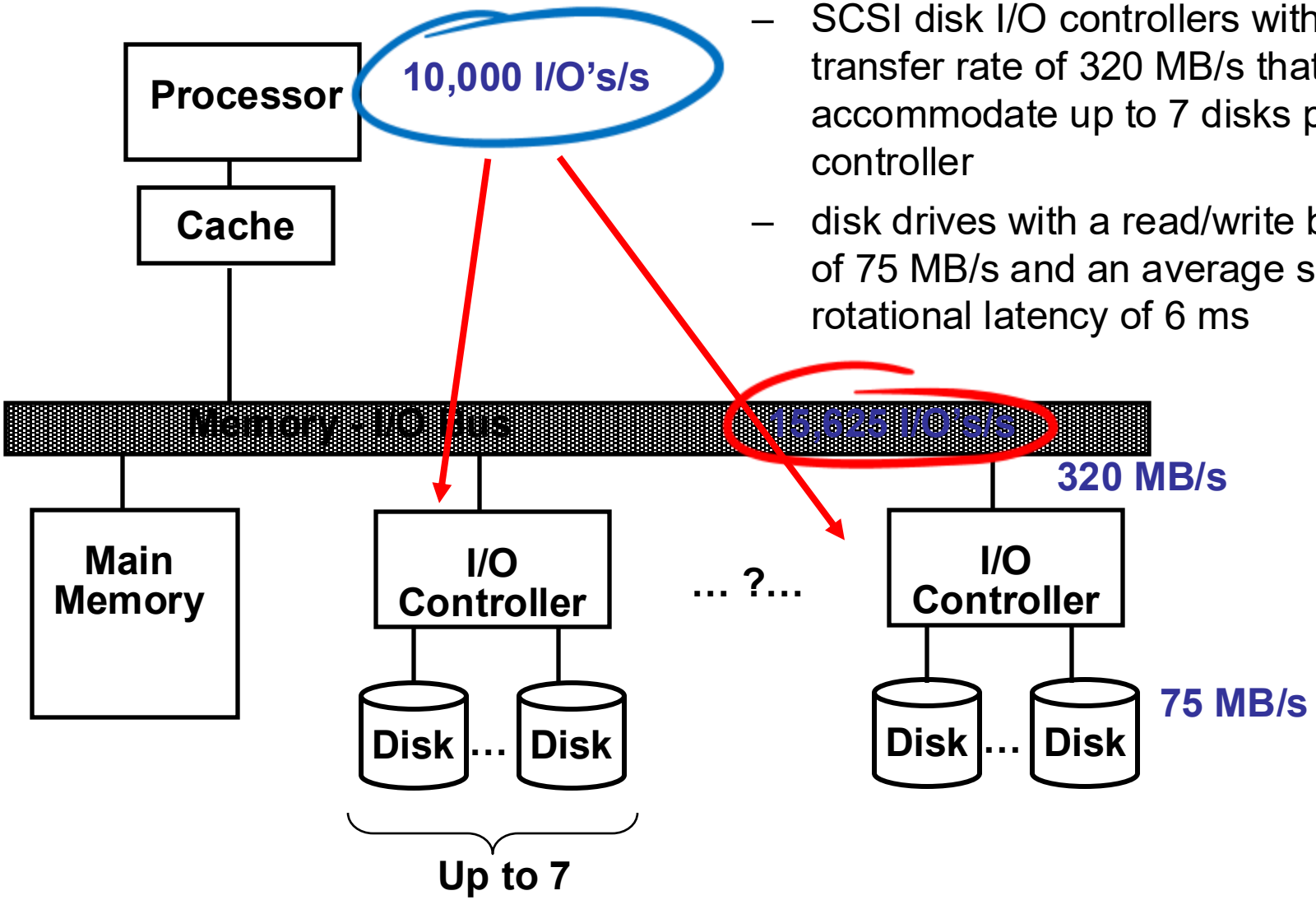
1. what is the maximum sustainable I/O rate?

2. what is the number of disks and SCSI controllers required to achieve that rate?

Disk I/O System Example

Suppose

- SCSI disk I/O controllers with a DMA transfer rate of 320 MB/s that can accommodate up to 7 disks per controller
- disk drives with a read/write bandwidth of 75 MB/s and an average seek plus rotational latency of 6 ms



So the processor is the bottleneck, not the bus

I/O System Performance Example, Con't

So the processor is the bottleneck, not the bus

- disk drives with a read/write bandwidth of 75 MB/s and an average seek plus rotational latency of 6 ms

Disk I/O read/write time = seek + rotational time + transfer time =~
6ms + 64KB/(75MB/s) = 6.9ms

Thus each disk can complete 1000ms/6.9ms or 146 I/O's per second. To saturate the processor requires 10,000 I/O's per second or
10,000/146 = 69 disks

To calculate the number of SCSI disk controllers, we need to know the average transfer rate per disk to ensure we can put the maximum of 7 disks per SCSI controller and that a disk controller won't saturate the memory-I/O bus during a DMA transfer

Disk transfer rate = (transfer size)/(transfer time) = 64KB/6.9ms = 9.56 MB/s

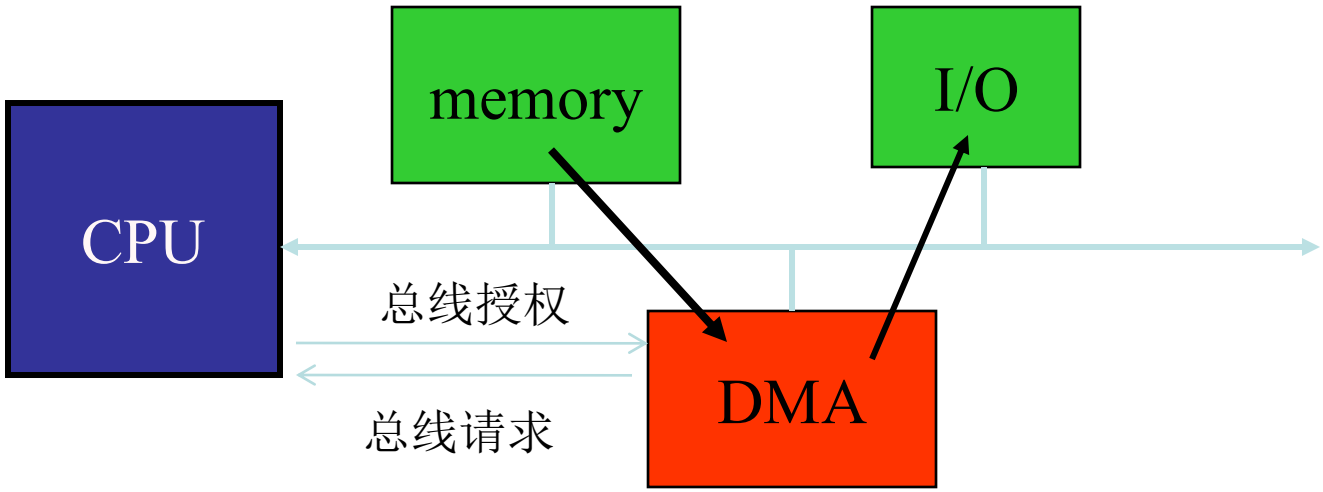
Thus 7 disks won't saturate either the SCSI controller (with a maximum transfer rate of 320 MB/s) or the memory-I/O bus (1000 MB/s). This means we will need ceiling(69/7) or 10 SCSI controllers.

I/O System Interconnect Issues

- A bus is a shared communication link (a single set of wires used to connect multiple subsystems) that needs to support a range of devices with widely varying latencies and data transfer rates
  - Advantages
    - Versatile – new devices can be added easily and can be moved between computer systems that use the same bus standard
    - Low cost – a single set of wires is shared in multiple ways
  - Disadvantages
    - Creates a communication bottleneck – bus bandwidth limits the maximum I/O throughput
- The maximum bus speed is largely limited by
  - The "length" of the bus
  - The number of devices on the bus
  - The need to support a wide range of devices with a widely varying latencies and data transfer rates.

Direct memory access (DMA)

- DMA provides parallelism on bus by controlling transfers without CPU.

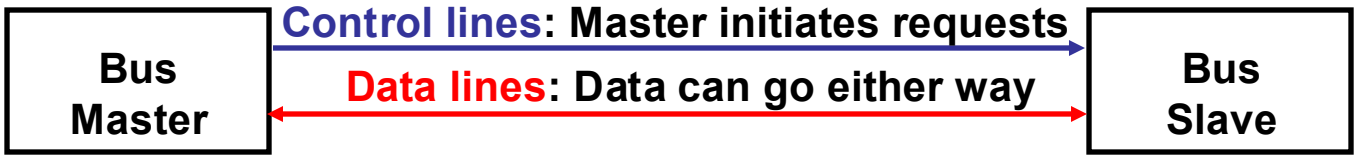


DMA operation

- CPU sets up DMA transfer:
  - Start address.
  - Length.
  - Transfer block length.
  - Style of transfer.

- DMA controller performs transfer, signals when done:
  - Cycle-stealing. 周期性交替
  - Priority. 优先级顺序交替

Bus Characteristics



- Control lines
  - Signal requests and acknowledgments
  - Indicate what type of information is on the data lines
- Data lines
  - Data, addresses, and complex commands
- Bus transaction consists of
  - Master issuing the command (and address) – request
  - Slave receiving (or sending) the data – action
  - Defined by what the transaction does to memory
    - Input – inputs data from the I/O device to the main memory
    - Output – outputs data from the main memory to the I/O device

Types of Buses

- Processor-memory bus (proprietary)
  - Short and high speed
  - Matched to the memory system to maximize the memory-processor bandwidth
  - Optimized for cache block transfers

- I/O bus (industry standard, e.g., SCSI, USB, Firewire)
  - Usually is lengthy and slower
  - Needs to accommodate a wide range of I/O devices
  - Connects to the processor-memory bus or backplane bus

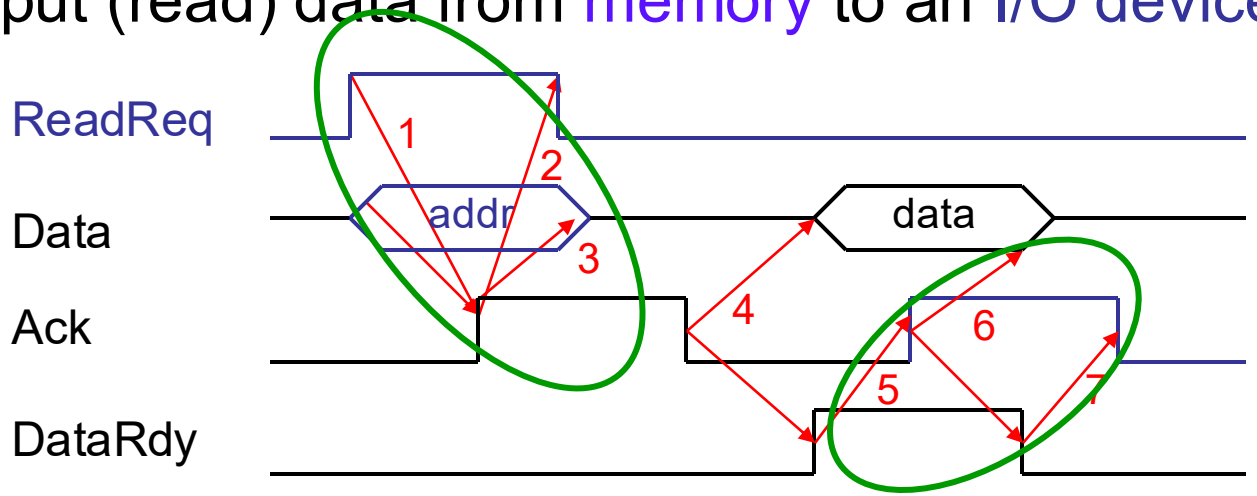
- Backplane bus (industry standard, e.g., ATA, PCIexpress)
  - The backplane is an interconnection structure within the chassis
  - allow processors, memory, and I/O devices to coexist on a single bus， cost advantage

Synchronous and Asynchronous Buses

- Synchronous bus (e.g., processor-memory buses)
  - Includes a clock in the control lines and has a fixed protocol for communication that is relative to the clock
  - Advantage: involves very little logic and can run very fast
  - Disadvantages:
    - Every device communicating on the bus must use same clock rate
    - To avoid clock skew, they cannot be long if they are fast
- Asynchronous bus (e.g., I/O buses)
  - It is not clocked, so requires a handshaking protocol and additional control lines (ReadReq, Ack, DataRdy)
  - Advantages:
    - Can accommodate a wide range of devices and device speeds
    - Can be lengthened without worrying about clock skew or synchronization problems
  - Disadvantage: slow(er)

Asynchronous Bus Handshaking Protocol

- Output (read) data from memory to an I/O device



First

I/O device signals a request by raising ReadReq and putting the addr on the data lines

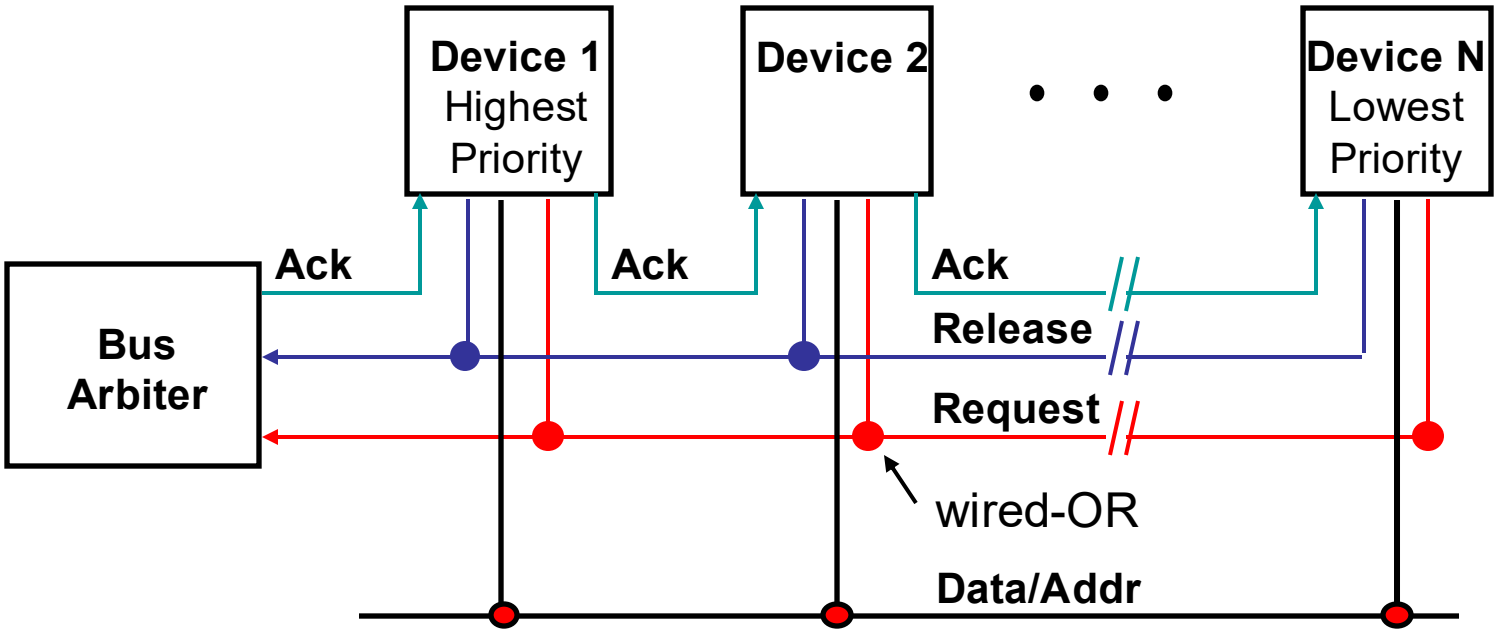
Then

1. Memory sees ReadReq, reads addr from data lines, and raises Ack
2. I/O device sees Ack and releases the ReadReq and data lines
3. Memory sees ReadReq go low and drops Ack
4. When memory has data ready, it places it on data lines and raises DataRdy
5. I/O device sees DataRdy, reads the data from data lines, and raises Ack
6. Memory sees Ack, releases the data lines, and drops DataRdy
7. I/O device sees DataRdy go low and drops Ack

The Need for Bus Arbitration

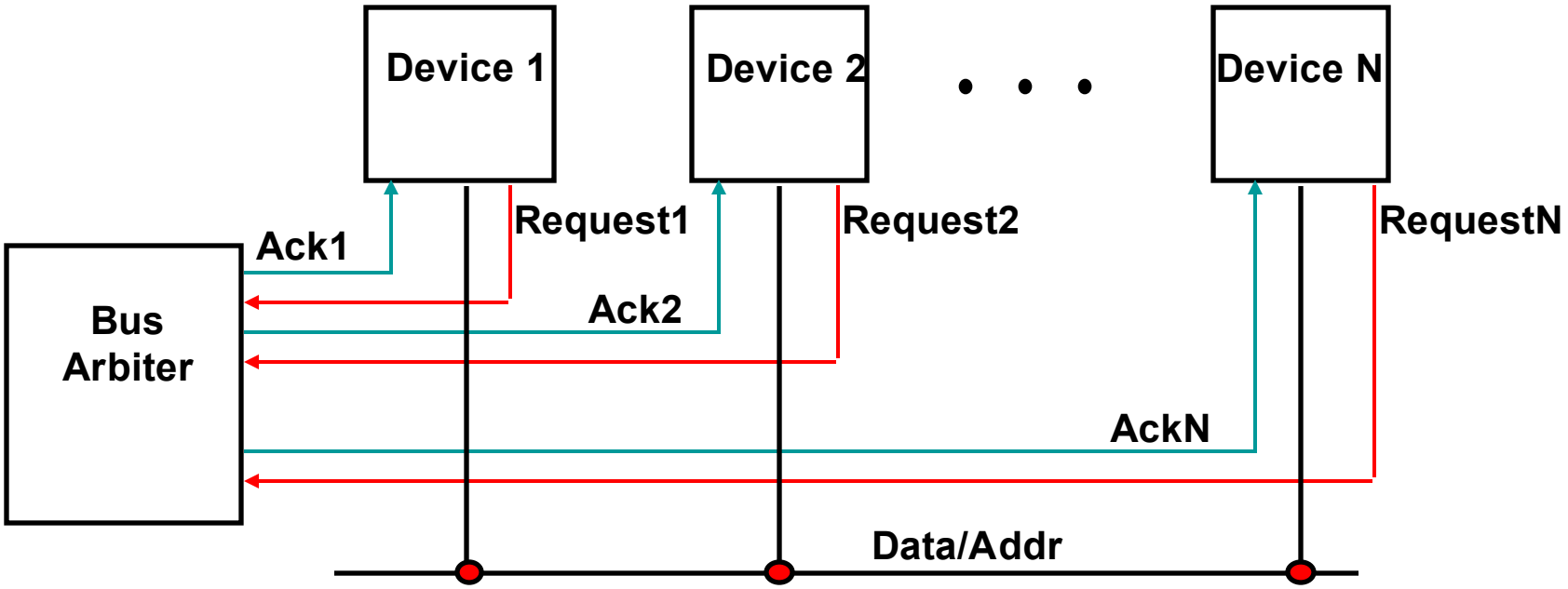
- Multiple devices may need to use the bus at the same time so must have a way to arbitrate multiple requests
- Bus arbitration schemes usually try to balance:
  - Bus priority – the highest priority device should be serviced first
  - Fairness – even the lowest priority device should never be completely locked out from the bus
- Bus arbitration schemes can be divided into four classes
  - Daisy chain arbitration – see next slide
  - Centralized, parallel arbitration – see next-next slide
  - Distributed arbitration by self-selection – each device wanting the bus places a code indicating its identity on the bus
  - Distributed arbitration by collision detection – device uses the bus when its not busy and if a collision happens (because some other device also decides to use the bus) then the device tries again later (Ethernet)

Daisy Chain Bus Arbitration



- Advantage: simple
- Disadvantages:
  - Cannot assure fairness – a low-priority device may be locked out indefinitely
  - Slower – the daisy chain grant signal limits the bus speed

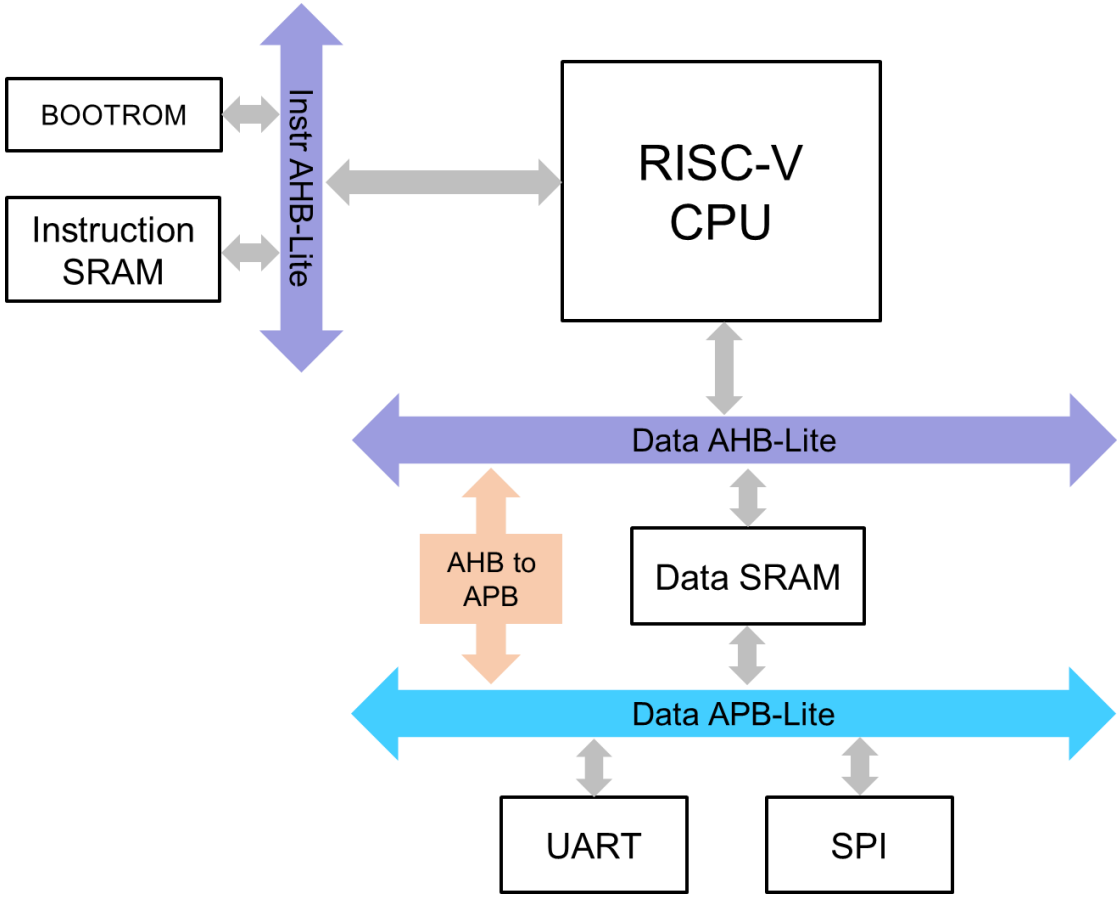
Centralized Parallel Arbitration



- Advantages: flexible, can assure fairness
- Disadvantages: more complicated arbiter hardware
- Used in essentially all processor-memory buses and in high-speed I/O buses

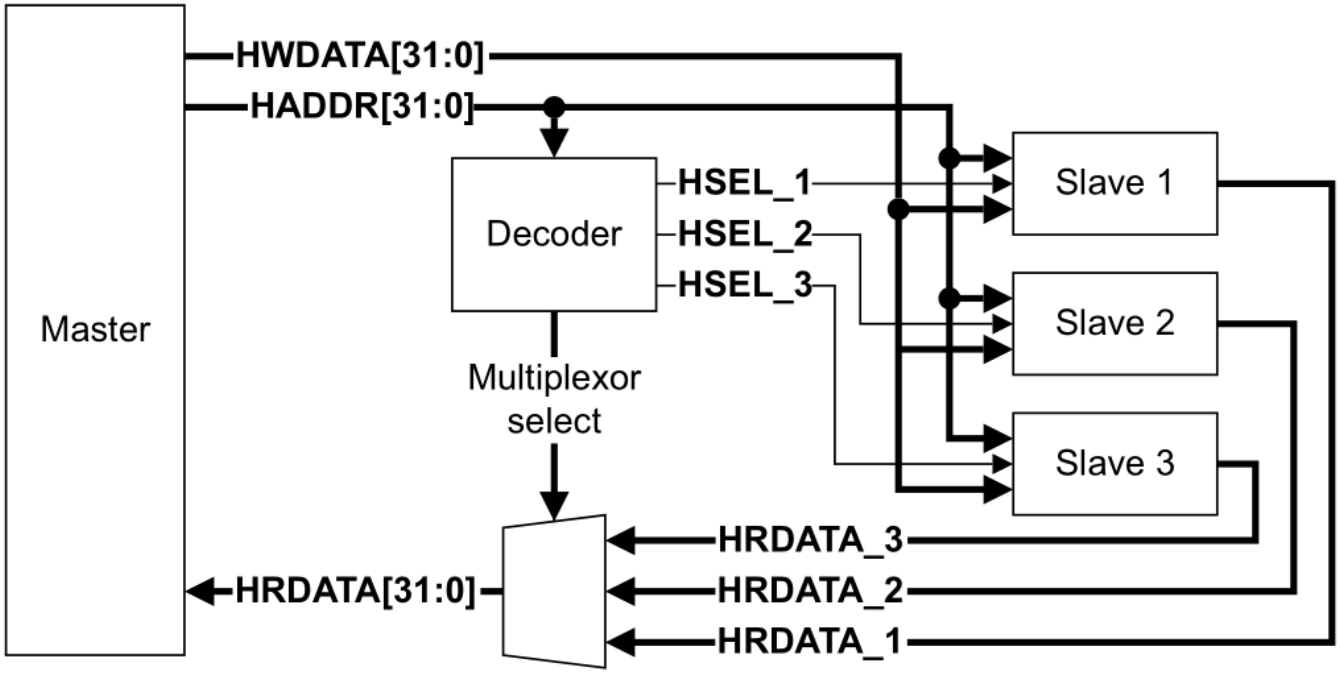
An SoC example with AHB-Lite bus

- RISC-V CPU with multiple bus
  - Instruction AHB-Lite bus
  - Data AHB-Lite bus
    - Low speed data APB-Lite bus



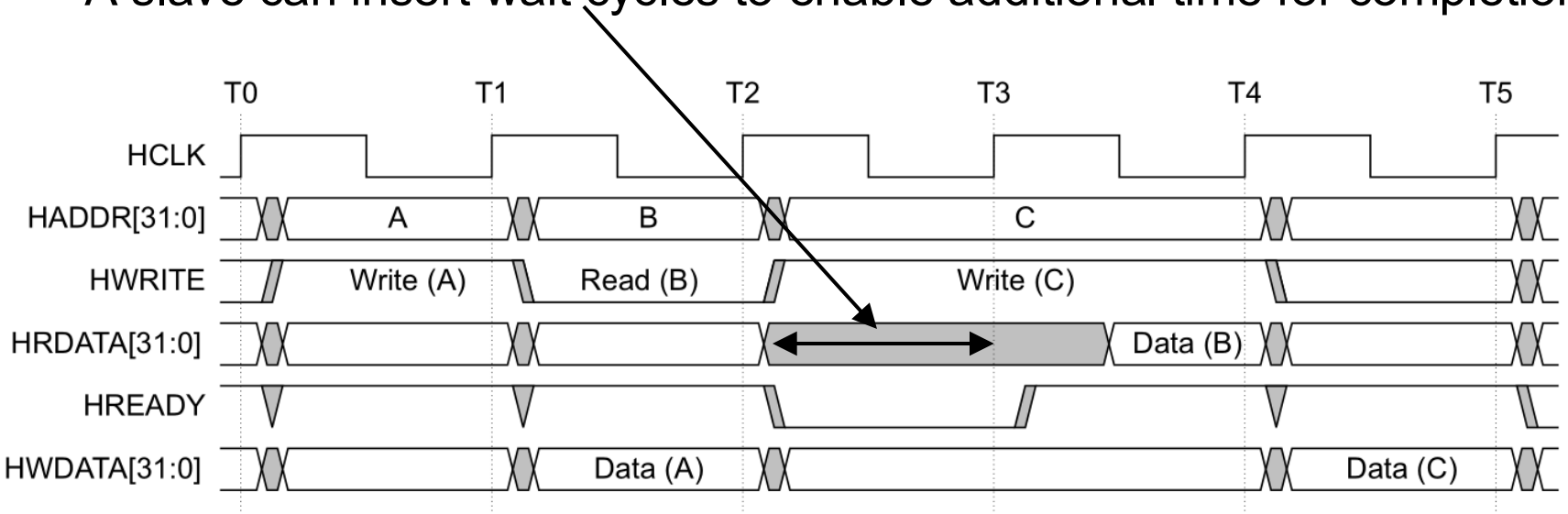
AHB-Lite bus

- AHB-Lite bus
  - One master, multiple slaves.
  - Each slave is allocated an address space, treated as a memory.
  - HADDR is sent to Decoder to choose one of the slaves.
  - Write HWDATA to the selected slave or read HRDATA_1/2/3 from it.



AHB-Lite bus

- AHB-Lite bus
  - High speed.
  - Read / write operation can be pipelined.
    - One cycle for r/w command, one cycle for r/w data.
    - The two cycles of adjacent r/w operations can be pipelined.
  - After pipeline, read / write operation can be finished in one clk cycle.
  - A slave can insert wait cycles to enable additional time for completion.



Reading

- Computer Organization and Design: The Hardware / Software Interface, 4th
- 6.5 ~ 6.8